

Software Requirements Specification for GamePhysics

Alex Halliwushka, Luthfi Mawarid, and Olu Owojaiye

November 3, 2025

Contents

1	Reference Material	3
1.1	Table of Units	3
1.2	Table of Symbols	3
1.3	Abbreviations and Acronyms	6
2	Introduction	6
2.1	Purpose of Document	7
2.2	Scope of Requirements	7
2.3	Characteristics of Intended Reader	7
2.4	Organization of Document	7
3	General System Description	8
3.1	System Context	8
3.2	User Characteristics	9
3.3	System Constraints	9
4	Specific System Description	9
4.1	Problem Description	9
4.1.1	Terminology and Definitions	9
4.1.2	Goal Statements	10
4.2	Solution Characteristics Specification	10
4.2.1	Assumptions	10
4.2.2	Theoretical Models	11
4.2.3	General Definitions	14
4.2.4	Data Definitions	17
4.2.5	Instance Models	32
4.2.6	Data Constraints	38
4.2.7	Properties of a Correct Solution	38

5	Requirements	39
5.1	Functional Requirements	39
5.2	Non-Functional Requirements	40
6	Likely Changes	40
7	Unlikely Changes	41
8	Off-The-Shelf Solutions	41
9	Traceability Matrices and Graphs	41
10	Values of Auxiliary Constants	48
11	References	48

1 Reference Material

This section records information for easy reference.

1.1 Table of Units

The unit system used throughout is SI (Système International d'Unités). In addition to the basic units, several derived units are also used. For each unit, the [Table of Units](#) lists the symbol, a description, and the SI name.

Table 1: Table of Units

Symbol	Description	SI Name
J	energy	joule
kg	mass	kilogram
m	length	metre
N	force	newton
rad	angle	radian
s	time	second

1.2 Table of Symbols

The symbols used in this document are summarized in the [Table of Symbols](#) along with their units. Throughout the document, symbols in bold will represent vectors, and scalars otherwise. The symbols are listed in alphabetical order. For vector quantities, the units shown are for each component of the vector.

Table 2: Table of Symbols

Symbol	Description	Units
$a(t)$	Linear acceleration	$\frac{m}{s^2}$
$\mathbf{a}(t)$	Acceleration	$\frac{m}{s^2}$
$\mathbf{a}(t)_j$	J-Th Body's Acceleration	$\frac{m}{s^2}$
C_R	Coefficient of restitution	—
d_j	Distance Between the J-Th Particle and the Axis of Rotation	m
$\mathbf{d}$	Distance between the center of mass of the rigid bodies	m
$\hat{\mathbf{d}}$	Unit vector directed from the center of the large mass to the center of the smaller mass	m

Continued on next page

Table 2: Table of Symbols (Continued)

Symbol	Description	Units
$\ \mathbf{d}\ $	Euclidean norm of the distance between the center of mass of two bodies	m
$\ \mathbf{d}\ ^2$	Squared distance	m^2
$\mathbf{F}$	Force	N
$\mathbf{F}_1$	Force exerted by the first body (on another body)	N
$\mathbf{F}_2$	Force exerted by the second body (on another body)	N
$\mathbf{F}_g$	Force of gravity	N
$\mathbf{F}_j$	Force Applied to the J-Th Body at Time T	N
G	Gravitational constant	$\frac{\text{m}^3}{\text{kg s}^2}$
$\mathbf{g}$	Gravitational acceleration	$\frac{\text{m}}{\text{s}^2}$
h	Height	m
$\mathbf{I}$	Moment of inertia	kg m^2
$\mathbf{I}_A$	Moment of Inertia of Rigid Body A	kg m^2
$\mathbf{I}_B$	Moment of Inertia of Rigid Body B	kg m^2
$\mathbf{J}$	Impulse (vector)	Ns
j	Impulse (scalar)	Ns
KE	Kinetic energy	J
L	Length	m
M	Mass of the Larger Rigid Body	kg
m	Mass	kg
m_1	Mass of the first body	kg
m_2	Mass of the second body	kg
m_A	Mass of Rigid Body A	kg
m_B	Mass of Rigid Body B	kg
m_j	Mass of the J-Th Particle	kg
m_T	Total Mass of the Rigid Body	kg
$\mathbf{n}$	Collision normal vector	m
$\ \mathbf{n}\ $	Length of the normal vector	m
PE	Potential energy	J
$\mathbf{p}(t)$	Position	m

Continued on next page

Table 2: Table of Symbols (Continued)

Symbol	Description	Units
$\mathbf{p}(t)_{\text{CM}}$	Center of Mass	m
$\mathbf{p}(t)_j$	Position Vector of the J-Th Particle	m
$\mathbf{r}$	Position vector	m
t	Time	s
t_c	Denotes the time at collision	s
$u(t)$	Linear displacement	m
$\mathbf{u}$	Displacement	m
$\ \mathbf{u}_{\text{AP}}^* \mathbf{n}\ $	Length of the Perpendicular Vector to the Contact Displacement Vector of Rigid Body A	m
$\ \mathbf{u}_{\text{BP}}^* \mathbf{n}\ $	Length of the Perpendicular Vector to the Contact Displacement Vector of Rigid Body B	m
$\mathbf{u}_{\text{OB}}$	Displacement vector between the origin and point B	m
$v(t)$	Linear velocity	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)$	Velocity	$\frac{\text{m}}{\text{s}}$
$\Delta \mathbf{v}$	Change in velocity	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)^{\text{AP}}$	Velocity of the Point of Collision P in Body A	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)^{\text{BP}}$	Velocity of the Point of Collision P in Body B	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)_1$	Velocity of the First Body	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)_2$	Velocity of the Second Body	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)_A$	Velocity at Point A	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)_B$	Velocity at Point B	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)^{\text{AB}}_f$	Final Relative Velocity Between Rigid Bodies of A and B	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)^{\text{AB}}_i$	Initial Relative Velocity Between Rigid Bodies of A and B	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)_j$	Velocity of the J-Th Body	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)_O$	Velocity at Point Origin	$\frac{\text{m}}{\text{s}}$
α	Angular acceleration	$\frac{\text{rad}}{\text{s}^2}$
α_j	J-Th Body's Angular Acceleration	$\frac{\text{rad}}{\text{s}^2}$
θ	Angular displacement	rad
π	Ratio of circumference to diameter for any circle	—

Continued on next page

Table 2: Table of Symbols (Continued)

Symbol	Description	Units
τ	Torque	Nm
τ_j	Torque applied to the j-th body	Nm
ω	Angular velocity	$\frac{\text{rad}}{\text{s}}$
ϕ	Orientation	rad

1.3 Abbreviations and Acronyms

Table 3: Abbreviations and Acronyms

Abbreviation	Full Form
2D	Two-Dimensional
3D	Three-Dimensional
A	Assumption
CM	Centre of Mass
DD	Data Definition
GD	General Definition
GS	Goal Statement
IM	Instance Model
LC	Likely Change
ODE	Ordinary Differential Equation
R	Requirement
RefBy	Referenced by
Refname	Reference Name
SRS	Software Requirements Specification
TM	Theoretical Model
UC	Unlikely Change
Uncert.	Typical Uncertainty

2 Introduction

Due to the rising cost of developing video games, developers are looking for ways to save time and money for their projects. Using an open source physics library that is reliable and

free will cut down development costs and lead to better quality products. The document describes the program based on the original, manually created version of [GamePhysics](#).

The following section provides an overview of the Software Requirements Specification (SRS) for GamePhysics. This section explains the purpose of this document, the scope of the requirements, the characteristics of the intended reader, and the organization of the document.

2.1 Purpose of Document

The primary purpose of this document is to record the requirements of GamePhysics. Goals, assumptions, theoretical models, definitions, and other model derivation information are specified, allowing the reader to fully understand and verify the purpose and scientific basis of GamePhysics. With the exception of [system constraints](#), this SRS will remain abstract, describing what problem is being solved, but not how to solve it.

This document will be used as a starting point for subsequent development phases, including writing the design specification and the software verification and validation plan. The design document will show how the requirements are to be realized, including decisions on the numerical algorithms and programming environment. The verification and validation plan will show the steps that will be used to increase confidence in the software documentation and the implementation. Although the SRS fits in a series of documents that follow the so-called waterfall model, the actual development process is not constrained in any way. Even when the waterfall model is not followed, as Parnas and Clements point out [8], the most logical way to present the documentation is still to “fake” a rational design process.

2.2 Scope of Requirements

The scope of the requirements includes the physical simulation of 2D rigid bodies acted on by forces.

2.3 Characteristics of Intended Reader

Reviewers of this documentation should have an understanding of rigid body dynamics and high school calculus. The users of GamePhysics can have a lower level of expertise, as explained in [Sec:User Characteristics](#).

2.4 Organization of Document

The organization of this document follows the template for an SRS for scientific computing software proposed by [6], [11], [12], and [10]. The presentation follows the standard pattern of presenting goals, theories, definitions, and assumptions. For readers that would like a more bottom up approach, they can start reading the [instance models](#) and trace back to find any additional information they require.

The **goal statements** are refined to the theoretical models and the **theoretical models** to the **instance models**.

3 General System Description

This section provides general information about the system. It identifies the interfaces between the system and its environment, describes the user characteristics, and lists the system constraints.

3.1 System Context

Fig:sysCtxDiag shows the system context. A circle represents an entity external to the software, the user in this case. A rectangle represents the software system itself (GamePhysics). Arrows are used to show the data flow between the system and its environment.

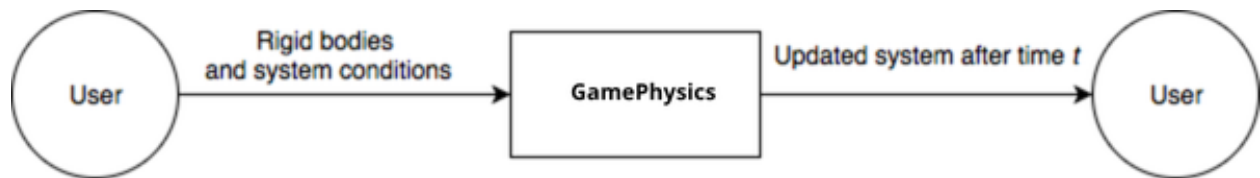


Figure 1: System Context

The interaction between the product and the user is through an application programming interface. The responsibilities of the user and the system are as follows:

- User Responsibilities
 - Provide initial conditions of the physical state of the simulation, rigid bodies present, and forces applied to them.
 - Ensure application programming interface use complies with the user guide.
 - Ensure required **software assumptions** are appropriate for any particular problem the software addresses.
- GamePhysics Responsibilities
 - Determine if the inputs and simulation state satisfy the required **physical and system constraints**.
 - Calculate the new state of all rigid bodies within the simulation at each simulation step.
 - Provide updated physical state of all rigid bodies at the end of a simulation step.

3.2 User Characteristics

The end user of GamePhysics should have an understanding of first year programming concepts and an understanding of high school physics.

3.3 System Constraints

There are no system constraints.

4 Specific System Description

This section first presents the problem description, which gives a high-level view of the problem to be solved. This is followed by the solution characteristics specification, which presents the assumptions, theories, and definitions that are used.

4.1 Problem Description

A system is needed to simulate 2D rigid body physics for use in game development in a simple, lightweight, fast, and portable manner, which will allow for the production of higher quality products. Creating a gaming physics library is a difficult task. Games need physics libraries that simulate objects acting under various physical conditions, while simultaneously being fast and efficient enough to work in soft real-time during the game. Developing a physics library from scratch takes a long period of time and is very costly, presenting barriers of entry which make it difficult for game developers to include physics in their products. There are a few free, open source and high quality **physics libraries** available to be used for consumer products.

4.1.1 Terminology and Definitions

This subsection provides a list of terms that are used in the subsequent sections and their meaning, with the purpose of reducing ambiguity and making it easier to correctly understand the requirements.

- Rigid body: A solid body in which deformation is neglected.
- Elasticity: The ratio of the relative velocities of two colliding objects after and before a collision.
- Centre of mass: The mean location of the distribution of mass of the object.
- Cartesian coordinate system: A coordinate system that specifies each point uniquely in a plane by a set of numerical coordinates, which are the signed distances to the point from two fixed perpendicular oriented lines, measured in the same unit of length (from [2]).

- Right-handed coordinate system: A coordinate system where the positive z-axis comes out of the screen.
- line: An interval between two points (from [5]).
- point: An exact location, it has no size, only position (from [9]).
- damping: An influence within or upon an oscillatory system that has the effect of reducing, restricting or preventing its oscillations (from [4]).

4.1.2 Goal Statements

Given the kinematic properties, and forces (including any collision forces) applied on a set of rigid bodies, the goal statements are:

Determine-Linear-Properties: Determine their new positions and velocities over a period of time.

Determine-Angular-Properties: Determine their new orientations and angular velocities over a period of time.

4.2 Solution Characteristics Specification

The instance models that govern GamePhysics are presented in the [Instance Model Section](#). The information to understand the meaning of the instance models and their derivation is also presented, so that the instance models can be verified.

4.2.1 Assumptions

This section simplifies the original problem and helps in developing the theoretical models by filling in the missing information for the physical system. The assumptions refine the scope by providing more detail.

objectTy: All objects are rigid bodies. (RefBy: [GD:impulse](#), [IM:col2D](#), [IM:rotMot](#), [IM:transMot](#), [DD:chasesThm](#), [DD:reVeInColl](#), [DD:potEnergy](#), [DD:ctrOfMass](#), [DD:momentOfInertia](#), [DD:linVel](#), [DD:linDisp](#), [DD:linAcc](#), [DD:kEnergy](#), [DD:impulseV](#), [DD:angVel](#), [DD:angDisp](#), and [DD:angAccel](#).)

objectDimension: All objects are 2D. (RefBy: [TM:NewtonSecLawRotMot](#), [GD:impulse](#), [IM:col2D](#), [IM:rotMot](#), [IM:transMot](#), [DD:potEnergy](#), [DD:kEnergy](#), [DD:angVel](#), [DD:angDisp](#), and [DD:angAccel](#).)

coordinateSystemTy: The library uses a Cartesian coordinate system.

axesDefined: The axes are defined using right-handed coordinate system. (RefBy: [GD:impulse](#), [IM:col2D](#), and [IM:rotMot](#).)

collisionType: All rigid bodies collisions are vertex-to-edge collisions. (RefBy: [GD:impulse](#), [IM:col2D](#), and [LC:Expanded-Collisions](#).)

dampingInvolvement: There is no damping involved throughout the simulation and this implies that there are no friction forces. (RefBy: [IM:col2D](#), [IM:transMot](#), [LC:Include-Dampening](#), [DD:potEnergy](#), and [DD:kEnergy](#).)

constraintsAndJointsInvolvement: There are no constraints and joints involved throughout the simulation. (RefBy: [IM:col2D](#), [IM:transMot](#), and [LC:Include-Joints-Constraints](#).)

4.2.2 Theoretical Models

This section focuses on the general equations and laws that GamePhysics is based on.

Refname	TM:NewtonSecLawMot
Label	Newton's second law of motion
Equation	$\mathbf{F} = m \mathbf{a}(t)$
Description	$\mathbf{F}$ is the force (N) m is the mass (kg) $\mathbf{a}(t)$ is the acceleration ($\frac{\text{m}}{\text{s}^2}$)
Notes	The net force $\mathbf{F}$ on a body is proportional to the acceleration $\mathbf{a}(t)$ of the body, where m denotes the mass of the body as the constant of proportionality.
Source	—
RefBy	IM:transMot

Refname	TM:NewtonThirdLawMot
Label	Newton's third law of motion
Equation	$\mathbf{F}_1 = -\mathbf{F}_2$
Description	$\mathbf{F}_1$ is the force exerted by the first body (on another body) (N) $\mathbf{F}_2$ is the force exerted by the second body (on another body) (N)
Notes	Every action has an equal and opposite reaction. In other words, the force $\mathbf{F}_1$ exerted on the second rigid body by the first is equal in magnitude and in the opposite direction to the force $\mathbf{F}_2$ exerted on the first rigid body by the second.
Source	—
RefBy	

Refname	TM:UniversalGravLaw
Label	Newton's law of universal gravitation
Equation	$\mathbf{F} = G \frac{m_1 m_2}{\ \mathbf{d}\ ^2} \hat{\mathbf{d}} = G \frac{m_1 m_2}{\ \mathbf{d}\ ^2} \frac{\mathbf{d}}{\ \mathbf{d}\ }$
Description	$\mathbf{F}$ is the force (N) G is the gravitational constant ($\frac{\text{m}^3}{\text{kg s}^2}$) m_1 is the mass of the first body (kg) m_2 is the mass of the second body (kg) $\ \mathbf{d}\$ is the Euclidean norm of the distance between the center of mass of two bodies (m) $\hat{\mathbf{d}}$ is the unit vector directed from the center of the large mass to the center of the smaller mass (m) $\mathbf{d}$ is the distance between the center of mass of the rigid bodies (m)
Notes	Two rigid bodies in the universe attract each other with a force $\mathbf{F}$ that is directly proportional to the product of their masses, m_1 and m_2 , and inversely proportional to the squared distance $\ \mathbf{d}\ ^2$ between them.
Source	—
RefBy	GD:accelGravity

Refname	TM:NewtonSecLawRotMot
Label	Newton's second law for rotational motion
Equation	$\boldsymbol{\tau} = \mathbf{I} \alpha$
Description	$\boldsymbol{\tau}$ is the torque (Nm) $\mathbf{I}$ is the moment of inertia (kgm^2) α is the angular acceleration ($\frac{\text{rad}}{\text{s}^2}$)
Notes	The net torque $\boldsymbol{\tau}$ on a rigid body is proportional to its angular acceleration α, where $\mathbf{I}$ denotes the moment of inertia of the rigid body as the constant of proportionality. We also assume that all rigid bodies involved are two-dimensional (from A:objectDimension).
Source	–
RefBy	IM:rotMot

4.2.3 General Definitions

This section collects the laws and equations that will be used to build the instance models.

Refname	GD:accelGravity
Label	Acceleration due to gravity
Units	$\frac{\text{m}}{\text{s}^2}$
Equation	$\mathbf{g} = -\frac{GM}{\ \mathbf{d}\ ^2} \hat{\mathbf{d}}$
Description	$\mathbf{g}$ is the gravitational acceleration ($\frac{\text{m}}{\text{s}^2}$) G is the gravitational constant ($\frac{\text{m}^3}{\text{kg s}^2}$) M is the mass of the larger rigid body (kg) $\ \mathbf{d}\$ is the Euclidean norm of the distance between the center of mass of two bodies (m) $\hat{\mathbf{d}}$ is the unit vector directed from the center of the large mass to the center of the smaller mass (m)
Notes	If one of the masses is much larger than the other, it is convenient to define a gravitational field around the larger mass as shown above. The negative sign in the equation indicates that the force is an attractive force.
Source	Definition of Gravitational Acceleration
RefBy	IM:transMot

Detailed derivation of gravitational acceleration: From [Newton's law of universal gravitation](#), we have:

$$\mathbf{F} = \frac{G m_1 m_2}{\|\mathbf{d}\|^2} \hat{\mathbf{d}}$$

The above equation governs the gravitational attraction between two bodies. Suppose that one of the bodies is significantly more massive than the other, so that we concern ourselves with the force the massive body exerts on the lighter body. Further, suppose that the Cartesian coordinate system is chosen such that this force acts on a line which lies along one of the principal axes. Then our unit vector directed from the center of the large mass to the center of the smaller mass $\hat{\mathbf{d}}$ for the x or y axes is:

$$\hat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|}$$

Given the above assumptions, let M and m be the mass of the massive and light body respectively. Equating $\mathbf{F}$ above with Newton's second law for the force experienced by the light body, we get:

$$\mathbf{F}_{\mathbf{g}} = G \frac{Mm}{\|\mathbf{d}\|^2} \hat{\mathbf{d}} = m \mathbf{g}$$

where $\mathbf{g}$ is the gravitational acceleration. Dividing the above equation by m , we have:

$$G \frac{M}{\|\mathbf{d}\|^2} \hat{\mathbf{d}} = \mathbf{g}$$

and thus the negative sign indicates that the force is an attractive force:

$$\mathbf{g} = -G \frac{M}{\|\mathbf{d}\|^2} \hat{\mathbf{d}}$$

Refname	GD:impulse
Label	Impulse for Collision
Units	Ns
Equation	$j = \frac{-(1+C_R) \mathbf{v}(t)_i^{AB} \cdot \mathbf{n}}{\left(\frac{1}{m_A} + \frac{1}{m_B}\right) \ \mathbf{n}\ ^2 + \frac{\ \mathbf{u}_{AP} * \mathbf{n}\ ^2}{\mathbf{I}_A} + \frac{\ \mathbf{u}_{BP} * \mathbf{n}\ ^2}{\mathbf{I}_B}}$
Description	j is the impulse (scalar) (Ns) C_R is the coefficient of restitution (Unitless) $\mathbf{v}(t)_i^{AB}$ is the initial relative velocity between rigid bodies of A and B ($\frac{m}{s}$) $\mathbf{n}$ is the collision normal vector (m) m_A is the mass of rigid body A (kg) m_B is the mass of rigid body B (kg) $\ \mathbf{n}\$ is the length of the normal vector (m) $\ \mathbf{u}_{AP} * \mathbf{n}\$ is the length of the perpendicular vector to the contact displacement vector of rigid body A (m) $\mathbf{I}_A$ is the moment of inertia of rigid body A (kgm^2) $\ \mathbf{u}_{BP} * \mathbf{n}\$ is the length of the perpendicular vector to the contact displacement vector of rigid body B (m) $\mathbf{I}_B$ is the moment of inertia of rigid body B (kgm^2)
Notes	All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension). A right-handed coordinate system is used (from A:axesDefined). All collisions are vertex-to-edge (from A:collisionType).
Source	Impulse for Collision Ref
RefBy	IM:col2D

4.2.4 Data Definitions

This section collects and defines all the data needed to build the instance models.

Refname	DD:ctrOfMass
Label	Center of Mass
Symbol	$\mathbf{p}(t)_{\text{CM}}$
Units	m
Equation	$\mathbf{p}(t)_{\text{CM}} = \frac{\sum m_j \mathbf{p}(t)_j}{m_T}$
Description	$\mathbf{p}(t)_{\text{CM}}$ is the Center of Mass (m) m_j is the mass of the j-th particle (kg) $\mathbf{p}(t)_j$ is the position vector of the j-th particle (m) m_T is the total mass of the rigid body (kg)
Notes	All bodies are assumed to be rigid (from A:objectTy).
Source	—
RefBy	IM:col2D and IM:transMot

Refname	DD:linDisp
Label	Linear displacement
Symbol	$u(t)$
Units	m
Equation	$u(t) = \frac{d\mathbf{p}(t)(t)}{dt}$
Description	$u(t)$ is the linear displacement (m) t is the time (s) $\mathbf{p}(t)$ is the position (m)
Notes	All bodies are assumed to be rigid (from A:objectTy).
Source	–
RefBy	IM:transMot

Refname	DD:linVel
Label	Linear velocity
Symbol	$v(t)$
Units	$\frac{\text{m}}{\text{s}}$
Equation	$v(t) = \frac{d\mathbf{u}(t)}{dt}$
Description	$v(t)$ is the linear velocity ($\frac{\text{m}}{\text{s}}$) t is the time (s) $\mathbf{u}$ is the displacement (m)
Notes	All bodies are assumed to be rigid (from A:objectTy).
Source	–
RefBy	IM:transMot

Refname	DD:linAcc
Label	Linear acceleration
Symbol	$a(t)$
Units	$\frac{\text{m}}{\text{s}^2}$
Equation	$a(t) = \frac{d\mathbf{v}(t)(t)}{dt}$
Description	$a(t)$ is the linear acceleration ($\frac{\text{m}}{\text{s}^2}$) t is the time (s) $\mathbf{v}(t)$ is the velocity ($\frac{\text{m}}{\text{s}}$)
Notes	All bodies are assumed to be rigid (from A:objectTy).
Source	–
RefBy	IM:transMot

Refname	DD:angDisp
Label	Angular displacement
Symbol	θ
Units	rad
Equation	$\theta = \frac{d\phi(t)}{dt}$
Description	θ is the angular displacement (rad) t is the time (s) ϕ is the orientation (rad)
Notes	All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension).
Source	—
RefBy	IM:rotMot

Refname	DD:angVel
Label	Angular velocity
Symbol	ω
Units	$\frac{\text{rad}}{\text{s}}$
Equation	$\omega = \frac{d\theta(t)}{dt}$
Description	ω is the angular velocity ($\frac{\text{rad}}{\text{s}}$) t is the time (s) θ is the angular displacement (rad)
Notes	All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension).
Source	—
RefBy	IM:rotMot

Refname	DD:angAccel
Label	Angular acceleration
Symbol	α
Units	$\frac{\text{rad}}{\text{s}^2}$
Equation	$\alpha = \frac{d\omega(t)}{dt}$
Description	α is the angular acceleration ($\frac{\text{rad}}{\text{s}^2}$) t is the time (s) ω is the angular velocity ($\frac{\text{rad}}{\text{s}}$)
Notes	All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension).
Source	—
RefBy	IM:rotMot

Refname	DD:chaslesThm
Label	Chasles' theorem
Symbol	$\mathbf{v}(t)_B$
Units	$\frac{\text{m}}{\text{s}}$
Equation	$\mathbf{v}(t)_B = \mathbf{v}(t)_O + \omega \times \mathbf{u}_{OB}$
Description	$\mathbf{v}(t)_B$ is the velocity at point B ($\frac{\text{m}}{\text{s}}$) $\mathbf{v}(t)_O$ is the velocity at point origin ($\frac{\text{m}}{\text{s}}$) ω is the angular velocity ($\frac{\text{rad}}{\text{s}}$) $\mathbf{u}_{OB}$ is the displacement vector between the origin and point B (m)
Notes	The linear velocity $\mathbf{v}(t)_B$ of any point B in a rigid body is the sum of the linear velocity $\mathbf{v}(t)_O$ of the rigid body at the origin (axis of rotation) and the resultant vector from the cross product of the rigid body's angular velocity ω and the displacement vector between the origin and point B $\mathbf{u}_{OB}$. All bodies are assumed to be rigid (from A:objectTy).
Source	[3]
RefBy	

Refname	DD:torque
Label	Torque
Symbol	$\boldsymbol{\tau}$
Units	Nm
Equation	$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$
Description	$\boldsymbol{\tau}$ is the torque (Nm) $\mathbf{r}$ is the position vector (m) $\mathbf{F}$ is the force (N)
Notes	The torque on a body measures the tendency of a force to rotate the body around an axis or pivot.
Source	—
RefBy	

Refname	DD:kEnergy
Label	Kinetic energy
Symbol	KE
Units	J
Equation	$KE = m \frac{\ \mathbf{v}(t)\ ^2}{2}$
Description	KE is the kinetic energy (J) m is the mass (kg) $\mathbf{v}(t)$ is the velocity ($\frac{\text{m}}{\text{s}}$)
Notes	Kinetic energy is the measure of the energy a body possesses due to its motion. All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension). No damping occurs during the simulation (from A:dampingInvolvement).
Source	–
RefBy	

Refname	DD:coeffRestitution
Label	Coefficient of restitution
Symbol	C_R
Units	Unitless
Equation	$C_R = - \left(\frac{\mathbf{v}(t)_f^{AB} \cdot \mathbf{n}}{\mathbf{v}(t)_i^{AB} \cdot \mathbf{n}} \right)$
Description	C_R is the coefficient of restitution (Unitless) $\mathbf{v}(t)_f^{AB}$ is the final relative velocity between rigid bodies of A and B ($\frac{m}{s}$) $\mathbf{n}$ is the collision normal vector (m) $\mathbf{v}(t)_i^{AB}$ is the initial relative velocity between rigid bodies of A and B ($\frac{m}{s}$)
Notes	The coefficient of restitution C_R determines the elasticity of a collision between two rigid bodies. $C_R = 1$ results in an elastic collision, $C_R < 1$ results in an inelastic collision, and $C_R = 0$ results in a totally inelastic collision.
Source	–
RefBy	

Refname	DD:reVeInColl
Label	Initial Relative Velocity Between Rigid Bodies of A and B
Symbol	$\mathbf{v}(t)_i^{AB}$
Units	$\frac{m}{s}$
Equation	$\mathbf{v}(t)_i^{AB} = \mathbf{v}(t)^{AP} - \mathbf{v}(t)^{BP}$
Description	$\mathbf{v}(t)_i^{AB}$ is the initial relative velocity between rigid bodies of A and B ($\frac{m}{s}$) $\mathbf{v}(t)^{AP}$ is the velocity of the point of collision P in body A ($\frac{m}{s}$) $\mathbf{v}(t)^{BP}$ is the velocity of the point of collision P in body B ($\frac{m}{s}$)
Notes	In a collision, the velocity of a rigid body A colliding with another rigid body B relative to that body $\mathbf{v}(t)_i^{AB}$ is the difference between the velocities of A and B at point P. All bodies are assumed to be rigid (from A:objectTy).
Source	—
RefBy	

Refname	DD:impulseV
Label	Impulse (vector)
Symbol	J
Units	Ns
Equation	$\mathbf{J} = m \Delta \mathbf{v}$
Description	J is the impulse (vector) (Ns) m is the mass (kg) $\Delta \mathbf{v}$ is the change in velocity ($\frac{\text{m}}{\text{s}}$)
Notes	An impulse (vector) J occurs when a force F acts over a body over an interval of time. All bodies are assumed to be rigid (from A:objectTy).
Source	–
RefBy	

Detailed derivation of impulse (vector): Newton's second law of motion states:

$$\mathbf{F} = m \mathbf{a}(t) = m \frac{d\mathbf{v}(t)}{dt}$$

Rearranging:

$$\int_{t_1}^{t_2} \mathbf{F} dt = m \left(\int_{\mathbf{v}(t)_1}^{\mathbf{v}(t)_2} 1 d\mathbf{v}(t) \right)$$

Integrating the right hand side:

$$\int_{t_1}^{t_2} \mathbf{F} dt = m \mathbf{v}(t)_2 - m \mathbf{v}(t)_1 = m \Delta \mathbf{v}$$

Refname	DD:potEnergy
Label	Potential energy
Symbol	PE
Units	J
Equation	$PE = m \mathbf{g} h$
Description	PE is the potential energy (J) m is the mass (kg) $\mathbf{g}$ is the gravitational acceleration ($\frac{\text{m}}{\text{s}^2}$) h is the height (m)
Notes	The potential energy of an object is the energy held by an object because of its position to other objects. All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension). No damping occurs during the simulation (from A:dampingInvolvement).
Source	–
RefBy	

Refname	DD:momentOfInertia
Label	Moment of inertia
Symbol	I
Units	kgm ²
Equation	$\mathbf{I} = \sum m_j d_j^2$
Description	I is the moment of inertia (kgm²) m_j is the mass of the j-th particle (kg) d_j is the distance between the j-th particle and the axis of rotation (m)
Notes	The moment of inertia I of a body measures how much torque is needed for the body to achieve angular acceleration about the axis of rotation. All bodies are assumed to be rigid (from A:objectTy).
Source	–
RefBy	

4.2.5 Instance Models

This section transforms the problem defined in the [problem description](#) into one which is expressed in mathematical terms. It uses concrete symbols defined in the [data definitions](#) to replace the abstract symbols in the models identified in [theoretical models](#) and [general definitions](#).

The goal [GS:Determine-Linear-Properties](#) is met by [IM:transMot](#) and [IM:col2D](#). The goal [GS:Determine-Angular-Properties](#) is met by [IM:rotMot](#) and [IM:col2D](#).

Refname	IM:transMot		
Label	J-Th Body's Acceleration		
Input	$\mathbf{v}(t)_j$, t , $\mathbf{g}$, $\mathbf{F}_j$, m_j		
Output	$\mathbf{a}(t)_j$		
Input Constraints	$\mathbf{v}(t)_j > 0$ $t > 0$ $\mathbf{g} > 0$ $\mathbf{F}_j > 0$ $m_j > 0$		
Output Constraints			
Equation	$\mathbf{a}(t)_j = \mathbf{g} + \frac{\mathbf{F}_j(t)}{m_j}$		
Description	$\mathbf{a}(t)_j$ is the j-th body's acceleration ($\frac{\text{m}}{\text{s}^2}$) $\mathbf{g}$ is the gravitational acceleration ($\frac{\text{m}}{\text{s}^2}$) $\mathbf{F}_j$ is the force applied to the j-th body at time t (N) t is the time (s) m_j is the mass of the j-th particle (kg)		
Notes	The above equation expresses the total acceleration of the rigid body j as the sum of gravitational acceleration (from GD:accelGravity) and acceleration due to applied force $\mathbf{F}_j(t)$ (from TM:NewtonSecLawMot). The resultant outputs are then obtained from this equation using DD:linDisp, DD:linVel, and DD:linAcc. The output of the instance model will be the functions of position and velocity over time that satisfy the ODE for the acceleration, with the given initial conditions for position and velocity. The motion is translational, so the position and velocity functions are for the centre of mass (from DD:ctrOfMass). All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension). It is currently assumed that no damping occurs during the simulation (from A:dampingInvolvement) and that no constraints are involved (from A:constraintsAndJointsInvolvement).		
Source	—		

Detailed derivation of j-th body's acceleration: We may calculate the total acceleration of rigid body j by calculating the derivative of it's velocity with respect to time (from [DD:linAcc](#)).

$$\alpha_j = \frac{d\mathbf{v}(t)_j(t)}{dt}$$

Performing the derivative, we obtain:

$$\mathbf{a}(t)_j = \mathbf{g} + \frac{\mathbf{F}_j(t)}{m_j}$$

Refname	IM:rotMot		
Label	J-Th Body's Angular Acceleration		
Input	$\omega, t, \boldsymbol{\tau}_j, \mathbf{I}$		
Output	α_j		
Input Constraints	$\omega > 0$ $t > 0$ $\boldsymbol{\tau}_j > 0$ $\mathbf{I} > 0$		
Output Constraints	$\alpha_j > 0$		
Equation	$\alpha_j = \frac{\boldsymbol{\tau}_j(t)}{\mathbf{I}}$		
Description	α_j is the j-th body's angular acceleration ($\frac{\text{rad}}{\text{s}^2}$) $\boldsymbol{\tau}_j$ is the torque applied to the j-th body (Nm) t is the time (s) $\mathbf{I}$ is the moment of inertia (kgm^2)		
Notes	The above equation for the total angular acceleration of the rigid body j is derived from TM:NewtonSecLawRotMot, and the resultant outputs are then obtained from this equation using DD:angDisp, DD:angVel, and DD:angAccel. All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension). A right-handed coordinate system is used (from A:axesDefined).		
Source	–		
RefBy			

Detailed derivation of j-th body's angular acceleration: We may calculate the total angular acceleration of rigid body j by calculating the derivative of its angular velocity with respect to time (from [DD:angAccel](#)).

$$\alpha_j = \frac{d\omega(t)}{dt}$$

Performing the derivative, we obtain:

$$\alpha_j = \frac{\tau_j(t)}{\mathbf{I}}$$

Refname	IM:col2D		
Label	Collisions on 2D rigid bodies		
Input	$t, j, m_A, \mathbf{n}$		
Output	t_c		
Input Constraints	$t > 0$ $j > 0$ $m_A > 0$ $\mathbf{n} > 0$		
Output Constraints	$t_c > 0$		
Equation	$\mathbf{v}(t)_A(t_c) = \mathbf{v}(t)_A(t) + \frac{j}{m_A} \mathbf{n}$		
Description	$\mathbf{v}(t)_A$ is the velocity at point A ($\frac{\text{m}}{\text{s}}$) t_c is the denotes the time at collision (s) t is the time (s) j is the impulse (scalar) (Ns) m_A is the mass of rigid body A (kg) $\mathbf{n}$ is the collision normal vector (m)		
Notes	The output of the instance model will be the functions of position, velocity, orientation, and angular acceleration over time that satisfy the equations for the velocity and angular acceleration, with the given initial conditions for position, velocity, orientation, and angular acceleration. The motion is translational, so the position, velocity, orientation, and angular acceleration functions are for the centre of mass (from DD:ctrOfMass). All bodies are assumed to be rigid (from A:objectTy) and two-dimensional (from A:objectDimension). A right-handed coordinate system is used (from A:axesDefined). All collisions are vertex-to-edge (from A:collisionType). It is currently assumed that no damping occurs during the simulation (from A:dampingInvolvement) and that no constraints are involved (from A:constraintsAndJointsInvolvement). j is defined in GD:impulse		
Source	—		

4.2.6 Data Constraints

The **Data Constraints Table** shows the data constraints on the input variables. The column for physical constraints gives the physical limitations on the range of values that can be taken by the variable. The uncertainty column provides an estimate of the confidence with which the physical quantities can be measured. This information would be part of the input if one were performing an uncertainty quantification exercise. The constraints are conservative to give the user of the model the flexibility to experiment with unusual situations. The column of typical values is intended to provide a feel for a common scenario.

Table 4: Input Data Constraints

Var	Physical Constraints	Software Constraints	Typical Value	Uncert.
C_R	$0 \leq C_R \leq 1$	—	0.8	10%
$\mathbf{F}$	—	—	98.1 N	10%
G	—	—	$66.743 \cdot 10^{-12} \frac{\text{m}^3}{\text{kg s}^2}$	10%
$\mathbf{I}$	$\mathbf{I} > 0$	—	74.5 kgm ²	10%
L	$L > 0$	—	44.2 m	10%
m	$m > 0$	—	56.2 kg	10%
$\mathbf{p}(t)$	—	—	0.412 m	10%
$\mathbf{v}(t)$	—	—	$2.51 \frac{\text{m}}{\text{s}}$	10%
τ	—	—	200 Nm	10%
ω	—	—	$2.1 \frac{\text{rad}}{\text{s}}$	10%
ϕ	—	$0 \leq \phi \leq 2\pi$	$\frac{\pi}{2}$ rad	10%

4.2.7 Properties of a Correct Solution

The **Data Constraints Table** shows the data constraints on the output variables. The column for physical constraints gives the physical limitations on the range of values that can be taken by the variable.

Table 5:

Out-
put
Data
Con-
straints

Var
$\mathbf{p}(t)$
$\mathbf{v}(t)$
ϕ
ω

5 Requirements

This section provides the functional requirements, the tasks and behaviours that the software is expected to complete, and the non-functional requirements, the qualities that the software is expected to exhibit.

5.1 Functional Requirements

This section provides the functional requirements, the tasks and behaviours that the software is expected to complete.

Simulation-Space: Create a space for all of the rigid bodies in the physical simulation to interact in.

Input-Initial-Conditions: Input the initial masses, velocities, orientations, angular velocities of, and forces applied on rigid bodies.

Input-Surface-Properties: Input the surface properties of the bodies such as friction or elasticity.

Verify-Physical_Constraints: Verify that the inputs satisfy the required physical constraints from the **solution characteristics specification**.

Calculate-Translation-Over-Time: Determine the positions and velocities over a period of time of the 2D rigid bodies acted upon by a force.

Calculate-Rotation-Over-Time: Determine the orientations and angular velocities over a period of time of the 2D rigid bodies.

Determine-Collisions: Determine if any of the rigid bodies in the space have collided.

Determine-Collision-Response-Over-Time: Determine the positions and velocities over a period of time of the 2D rigid bodies that have undergone a collision.

5.2 Non-Functional Requirements

This section provides the non-functional requirements, the qualities that the software is expected to exhibit.

Performance: The execution time for collision detection and collision resolution shall be comparable to an existing 2D physics library on the market (e.g. Pymunk).

Correctness: The output of simulation results shall be compared to an existing implementation like [Pymunk](#).

Usability: Software shall be easy to learn and use. Usability shall be measured by how long it takes a user to learn how to use the library to create a small program to simulate the movement of 2 bodies over time in space. Creating a program should take no less than 30 to 60 minutes for an intermediate to experienced programmer.

Understandability: Users of Tamias2D shall be able to learn the software with ease. Users shall be able to easily create a small program using the library. Creating a small program to simulate the movement of 2 bodies in space should take no less than 60 minutes.

Maintainability: If a likely change is made to the finished software, it will take at most 10% of the original development time, assuming the same development resources are available.

6 Likely Changes

This section lists the likely changes to be made to the software.

Variable-ODE-Solver: The internal ODE-solving algorithm used by the library may be changed in the future.

Expanded-Collisions: [A:collisionType](#) - The library may be expanded to deal with edge-to-edge and vertex-to-vertex collisions.

Include-Dampening: [A:dampingInvolvement](#) - The library may be expanded to include motion with damping.

Include-Joints-Constraints: [A:constraintsAndJointsInvolvement](#) - The library may be expanded to include joints and constraints.

7 Unlikely Changes

This section lists the unlikely changes to be made to the software.

Simulate-Rigid-Bodies: The goal of the system is to simulate the interactions of rigid bodies.

External-Input: There will always be a source of input data external to the software.

Cartesian-Coordinate-System: A Cartesian Coordinate system is used.

Objects-Rigid-Bodies: All objects are rigid bodies.

8 Off-The-Shelf Solutions

As mentioned in the [problem description](#), there already exist free open source game physics libraries. Similar 2D physics libraries are:

- [Box2D](#)
- [Nape Physics Engine](#)

Free open source 3D game physics libraries include:

- [Bullet](#)
- [Open Dynamics Engine](#)
- [Newton Game Dynamics](#)

9 Traceability Matrices and Graphs

The purpose of the traceability matrices is to provide easy references on what has to be additionally modified if a certain component is changed. Every time a component is changed, the items in the column of that component that are marked with an “X” should be modified as well. [Tab:TraceMatAvsA](#) shows the dependencies of the assumptions on each other. [Tab:TraceMatAvsAll](#) shows the dependencies of the data definitions, theoretical models, general definitions, instance models, requirements, likely changes, and unlikely changes on the assumptions. [Tab:TraceMatRefvsRef](#) shows the dependencies of the data definitions, theoretical models, general definitions, and instance models on each other. [Tab:TraceMatAllvsR](#) shows the dependencies of the requirements and goal statements on the data definitions, theoretical models, general definitions, and instance models.

Table 6: Traceability Matrix Showing the Conn

	A:objectTy	A:objectDimension	A:coordinateSystemTy
A:objectTy			
A:objectDimension			
A:coordinateSystemTy			
A:axesDefined			
A:collisionType			
A:dampingInvolvement			
A:constraintsAndJointsInvolvement			

Table 7: Traceability Matrix Showing t

	A:objectTy	A:objectDimension	A:coordinateSy
DD:ctrOfMass	X		
DD:linDisp	X		
DD:linVel	X		
DD:linAcc	X		
DD:angDisp	X	X	
DD:angVel	X	X	
DD:angAccel	X	X	
DD:chaslesThm	X		
DD:torque			
DD:kEnergy	X	X	
DD:coeffRestitution			
DD:reVeInColl	X		
DD:impulseV	X		
DD:potEnergy	X	X	
DD:momentOfInertia	X		
TM:NewtonSecLawMot			
TM:NewtonThirdLawMot			
TM:UniversalGravLaw			
TM:NewtonSecLawRotMot		X	

Table 7: Traceability Matrix Showing the Cor

	A:objectTy	A:objectDimension	A:coordinateSy
GD:accelGravity			
GD:impulse	X	X	
IM:transMot	X	X	
IM:rotMot	X	X	
IM:col2D	X	X	
FR:Simulation-Space			
FR:Input-Initial-Conditions			
FR:Input-Surface-Properties			
FR:Verify-Physical_Constraints			
FR:Calculate-Translation-Over-Time			
FR:Calculate-Rotation-Over-Time			
FR:Determine-Collisions			
FR:Determine-Collision-Response-Over-Time			
NFR:Performance			
NFR:Correctness			
NFR:Usability			
NFR:Understandability			
NFR:Maintainability			
LC:Variable-ODE-Solver			
LC:Expanded-Collisions			
LC:Include-Dampening			
LC:Include-Joints-Constraints			
UC:Simulate-Rigid-Bodies			
UC:External-Input			
UC:Cartesian-Coordinate-System			
UC:Objects-Rigid-Bodies			

	DD:ctrOfMass	DD:linDisp	DD:linVel	DD:linAcc	DD:angDis
DD:ctrOfMass					
DD:linDisp					
DD:linVel					
DD:linAcc					
DD:angDisp					
DD:angVel					
DD:angAccel					
DD:chaslesThm					
DD:torque					
DD:kEnergy					
DD:coeffRestitution					
DD:reVeInColl					
DD:impulseV					
DD:potEnergy					
DD:momentOfInertia					
TM:NewtonSecLawMot					
TM:NewtonThirdLawMot					
TM:UniversalGravLaw					
TM:NewtonSecLawRotMot					
GD:accelGravity					
GD:impulse					
IM:transMot	X	X	X	X	
IM:rotMot					X
IM:col2D	X				
	DD:ctrOfMass	DD:linDisp	DD:linVel	DD:linA	
GS:Determine-Linear-Properties					
GS:Determine-Angular-Properties					

FR:Simulation-Space
FR:Input-Initial-Conditions
FR:Input-Surface-Properties
FR:Verify-Physical_Constraints
FR:Calculate-Translation-Over-Time
FR:Calculate-Rotation-Over-Time
FR:Determine-Collisions
FR:Determine-Collision-Response-Over-Time
NFR:Performance
NFR:Correctness
NFR:Usability
NFR:Understandability
NFR:Maintainability

The purpose of the traceability graphs is also to provide easy references on what has to be additionally modified if a certain component is changed. The arrows in the graphs represent dependencies. The component at the tail of an arrow is depended on by the component at the head of that arrow. Therefore, if a component is changed, the components that it points to should also be changed. [Fig:TraceGraphAvsA](#) shows the dependencies of assumptions on each other. [Fig:TraceGraphAvsAll](#) shows the dependencies of data definitions, theoretical models, general definitions, instance models, requirements, likely changes, and unlikely changes on the assumptions. [Fig:TraceGraphRefvsRef](#) shows the dependencies of data definitions, theoretical models, general definitions, and instance models on each other. [Fig:TraceGraphAllvsR](#) shows the dependencies of requirements and goal statements on the data definitions, theoretical models, general definitions, and instance models. [Fig:TraceGraphAllvsAll](#) shows the dependencies of dependencies of assumptions, models, definitions, requirements, goals, and changes with each other.

A:assumpCAJI

Figure 2: TraceGraphAvsA

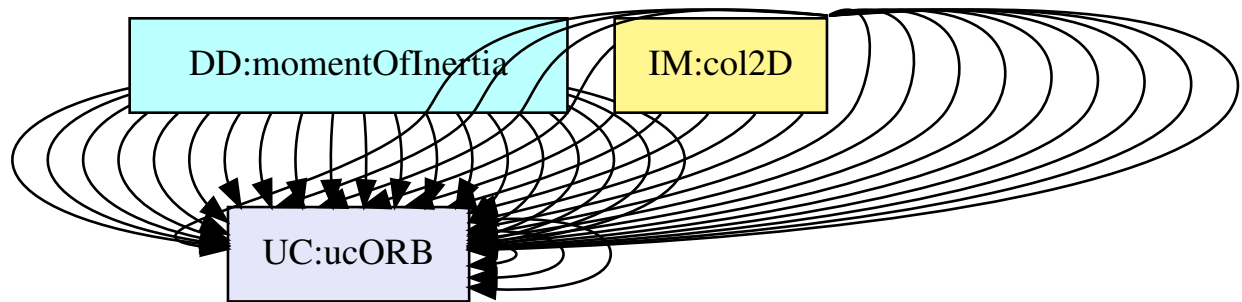


Figure 3: TraceGraphAvsAll

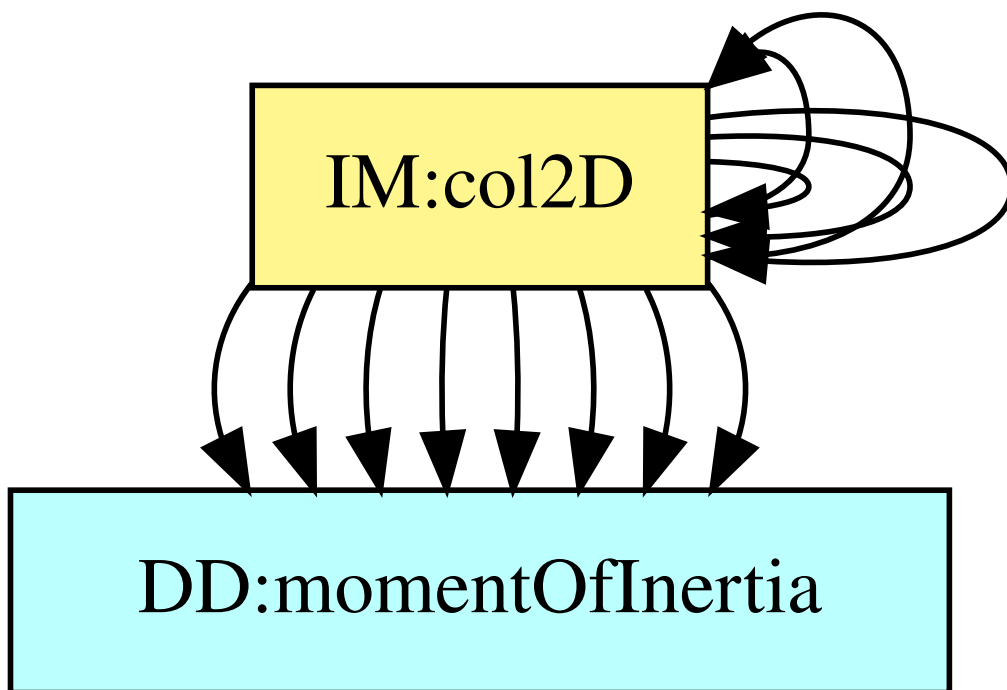


Figure 4: TraceGraphRefvsRef

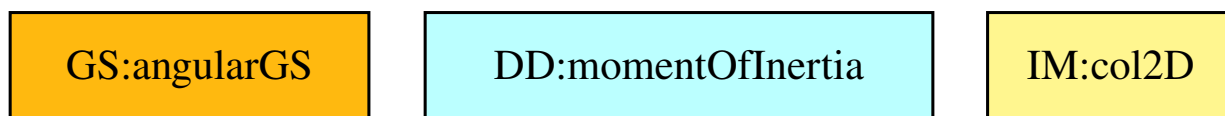


Figure 5: TraceGraphAllvsR

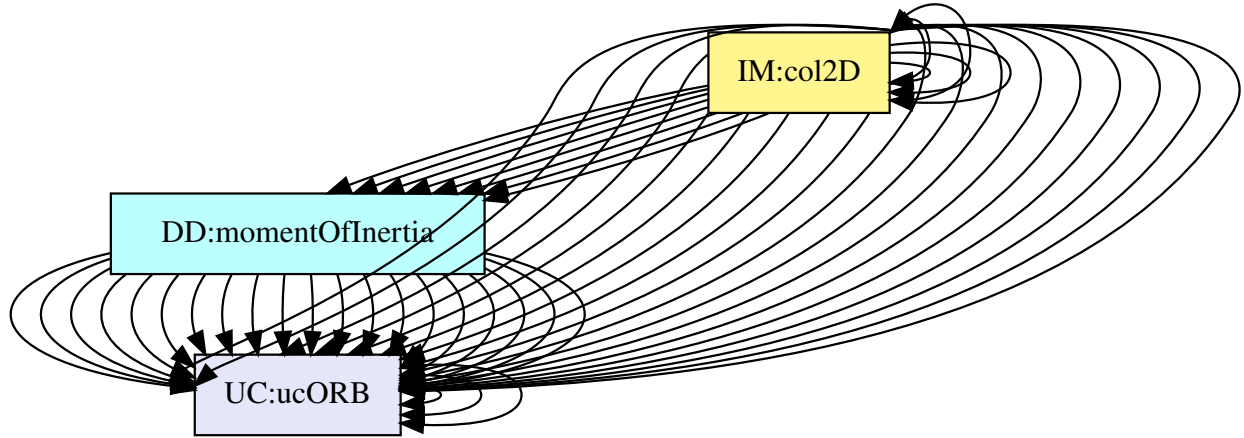


Figure 6: TraceGraphAllvsAll

For convenience, the following graphs can be found at the links below:

- [TraceGraphAvsA](#)
- [TraceGraphAvsAll](#)
- [TraceGraphRefvsRef](#)
- [TraceGraphAllvsR](#)
- [TraceGraphAllvsAll](#)

10 Values of Auxiliary Constants

There are no auxiliary constants.

11 References

- [1] J. Frederick Bueche. *Introduction to Physics for Scientists, Fourth Edition*. 1986.
- [2] Wikipedia Contributors. *Cartesian coordinate system*. https://en.wikipedia.org/wiki/Cartesian_coordinate_system. June 2019.
- [3] Wikipedia Contributors. *Chasles' theorem (kinematics)*. [https://en.wikipedia.org/wiki/Chasles'_theorem_\(kinematics\)](https://en.wikipedia.org/wiki/Chasles'_theorem_(kinematics)). Nov. 2018.
- [4] Wikipedia Contributors. *Damping*. https://en.wikipedia.org/wiki/Damping_ratio. July 2019.
- [5] The Editors of Encyclopaedia Britannica. *Line*. <https://www.britannica.com/science/line-mathematics>. June 2019.

- [6] Nirmitha Koothoor. “A Document Driven Approach to Certifying Scientific Computing Software”. MA thesis. Hamilton, ON, Canada: McMaster University, 2013.
- [7] David L. Parnas. “Designing Software for Ease of Extension and Contraction”. In: *ICSE '78: Proceedings of the 3rd international conference on Software engineering*. 1978, pp. 264–277.
- [8] David L. Parnas and P. C. Clements. “A rational design process: How and why to fake it”. In: *IEEE Transactions on Software Engineering* 12.2 (Feb. 1986), pp. 251–257.
- [9] Rod Pierce. *Point*. <https://www.mathsisfun.com/geometry/point.html>. May 2017.
- [10] W. Spencer Smith and Nirmitha Koothoor. “A Document-Driven Method for Certifying Scientific Computing Software for Use in Nuclear Safety Analysis”. In: *Nuclear Engineering and Technology* 48.2 (Apr. 2016), pp. 404–418.
- [11] W. Spencer Smith and Lei Lai. “A new requirements template for scientific computing”. In: *Proceedings of the First International Workshop on Situational Requirements Engineering Processes - Methods, Techniques and Tools to Support Situation-Specific Requirements Engineering Processes, SREP'05*. Ed. by PJ Agerfalk, N. Kraiem, and J. Ralyte. In conjunction with 13th IEEE International Requirements Engineering Conference, Paris, France, 2005, pp. 107–121.
- [12] W. Spencer Smith, Lei Lai, and Ridha Khedri. “Requirements Analysis for Engineering Computation: A Systematic Approach for Improving Software Reliability”. In: *Reliable Computing, Special Issue on Reliable Engineering Computation* 13.1 (Feb. 2007), pp. 83–107.
- [13] Greg Wilson et al. “Best Practices for Scientific Computing, 2013”. In: *PLoS Biol* 12.1 (2013).